# Data Reduction: Factor Analysis

Probability and Statistics — Week 10

# From Regression to Several Factors

Recall simple linear regression: given data points $(a_1, b_1), \ldots, (a_n, b_n)$, we fit a line

$$b = \mu + f \cdot a.$$

We generalise by allowing random error, for each observation $i = 1, \ldots, n$:

$$b_i = \mu + f \cdot a_i + \varepsilon_i, \qquad \varepsilon_i \sim N(0, 1) \text{ (e.g.)}.$$

With **two** explanatory factors:

$$\begin{aligned} b_1 &= \mu + f_1 a_{11} + f_2 a_{12} + \varepsilon_1, \\ b_2 &= \mu + f_1 a_{21} + f_2 a_{22} + \varepsilon_2, \\ &\vdots \end{aligned}$$

Generalising to $m$ factors:

$$b_i = \mu_i + f_1 a_{i1} + f_2 a_{i2} + \cdots + f_m a_{im} + \varepsilon_i, \qquad i = 1, \ldots, n.$$

# The General Factor Model

**Definition**

Factor Analysis (FA) and Principal Component Analysis (PCA) are statistical techniques applied to a (large) set of variables to try to reduce them into subsets of relatively independent variables. Such subsets contain variables that are correlated with one another, but largely independent of other subsets of variables, and are combined into factors (or components in PCA). The idea is to summarise patterns of correlations among observed variables and then use this information to reduce a large number of observed variables to a smaller number of factors.

In Factor Analysis, the subsets of variables are unobservable **latent variables** — we cannot measure them directly (e.g. intelligence or social class). We rename the notation ($b \to Y$, $f \to F$) to match the standard setup: if $Y_1, \dots, Y_n$ are the observed variables with means $\mu_1, \dots, \mu_n$, and $F_1, \dots, F_m$ are the $m$ hidden factors, we consider the centred observations $X_i = Y_i - \mu_i$.

**Definition**

$$\begin{aligned} X_1 = Y_1 - \mu_1 &= a_{11}F_1 + a_{12}F_2 + \cdots + a_{1m}F_m + \epsilon_1 \\ X_2 = Y_2 - \mu_2 &= a_{21}F_1 + a_{22}F_2 + \cdots + a_{2m}F_m + \epsilon_2 \\ \vdots \qquad & \qquad \vdots \qquad \qquad \vdots \\ X_n = Y_n - \mu_n &= a_{n1}F_1 + a_{n2}F_2 + \cdots + a_{nm}F_m + \epsilon_n, \end{aligned}$$

where $a_{ij}$ is the **factor loading** of the $i^{\text{th}}$ variable on the $j^{\text{th}}$ factor and $\epsilon_i$ is the error, or unique/specific factor, with mean 0 and specific variance $\psi_i$. Our aim is $m \ll n$ (data reduction).

# Worked Example (Ch-10.qmd)

**Example**

In an experiment, 200 primary school children were psychologically tested on: paragraph comprehension ($X_1$), sentence completion ($X_2$), word meaning ($X_3$), addition ($X_4$), counting ($X_5$). A factor analysis gives:

$$
\begin{aligned}
X_1 &= 0.81F_1 + 0.06F_2 + \epsilon_1 \\
X_2 &= 0.72F_1 + 0.08F_2 + \epsilon_2 \\
X_3 &= 0.91F_1 + 0.01F_2 + \epsilon_3 \\
X_4 &= 0.02F_1 + 0.69F_2 + \epsilon_4 \\
X_5 &= 0.11F_1 + 0.92F_2 + \epsilon_5
\end{aligned}
$$

Variables $X_1, X_2, X_3$ load highly on $F_1$ and low on $F_2$; $X_4, X_5$ load low on $F_1$ and high on $F_2$. This suggests $F_1 =$ **literacy skills** and $F_2 =$ **numeracy skills**.

## Matrix Formulation

In matrix notation, the whole system becomes

$$X = AF + \varepsilon,$$

where $X = (X_1, \dots, X_n)^T$, $A = (a_{ij})$, $F = (F_1, \dots, F_m)^T$, $\varepsilon = (\varepsilon_1, \dots, \varepsilon_n)^T$.

**Assumptions.** Without loss of generality, $\mathbb{E}(\varepsilon_i) = 0$ and $\mathbb{E}(F_j) = 0$ for all $i, j$ (any nonzero factor mean is absorbed into $\mu_i$). We assume $\mathrm{Var}(F_j) = 1$, that the factors are uncorrelated,

$$\mathrm{cov}(F_i, F_j) = \mathbb{E}(F_i F_j) = \begin{cases} 0, & i \neq j \\ 1, & i = j, \end{cases}$$

that $\varepsilon_i$ and $F_j$ are uncorrelated for all $i, j$, and that $\mathrm{cov}(\varepsilon_i, \varepsilon_j) = 0$ for $i \neq j$, $\mathrm{Var}(\varepsilon_i) = \psi_i$.

These assumptions give $\mathbb{E}(FF^T) = I$, $\mathbb{E}(F\varepsilon^T) = \mathbb{E}(\varepsilon F^T) = 0$, and $\mathbb{E}(\varepsilon\varepsilon^T) = \Psi = \mathrm{diag}(\psi_1, \dots, \psi_n)$ — facts used immediately below.

## The Covariance Structure $\Sigma = AA^T + \Psi$

Recall for random vectors, $\Sigma = \mathbb{E}\left[(X - \mathbb{E}(X))(X - \mathbb{E}(X))^T\right]$; since $\mathbb{E}(X) = 0$ here, $\Sigma = \mathbb{E}(XX^T)$.

Substituting $X = AF + \varepsilon$:

$$\begin{aligned}\mathbb{E}(XX^T) &= \mathbb{E}((AF + \epsilon)(AF + \epsilon)^T) \\ &= \mathbb{E}(AFF^TA^T) + A\mathbb{E}(F\epsilon^T) + \mathbb{E}(\epsilon F^T)A^T + \mathbb{E}(\epsilon\epsilon^T) \\ &= AIA^T + 0 + 0 + \Psi \\ &= AA^T + \Psi.\end{aligned}$$

The factors themselves drop out of the final formula.

Setting $R = AA^T = \Sigma - \Psi$ (the *adjusted* covariance matrix), $R$ is symmetric, so

$$R = VLV^T$$

with $V$ the eigenvectors of $R$ and $L$ diagonal of eigenvalues. Since $L$ is diagonal, $\sqrt{L}^T = \sqrt{L}$, so

$$R = V\sqrt{L}\sqrt{L}V^T = (V\sqrt{L})(V\sqrt{L})^T = AA^T \quad \Longrightarrow \quad A = V\sqrt{L}.$$

Once the eigenvectors/eigenvalues of $R$ are known, $A$ follows immediately.

## Non-Uniqueness of $A$: Rotation

$A$ is **not unique**: if $C$ is orthogonal ($CC^T = I$), then

$$\Sigma = AA^T + \Psi = AIA^T + \Psi = ACC^TA^T + \Psi = (AC)(AC)^T + \Psi,$$

so $AC$ is an equally valid loading matrix.

Correspondingly,

$$X = AF + \varepsilon = ACC^TF + \varepsilon = (AC)\tilde{F} + \varepsilon, \qquad \tilde{F} := C^TF,$$

i.e. any orthogonal transformation of $F$, compensated by the same transformation of $A$, gives an equally valid decomposition of $\Sigma$. This is the mathematical fact underlying **factor rotation**.

# Computational Illustration — Setup

We now make the theory above completely concrete with simulated exam marks for 10 students in 6 subjects (Math-1, Math-2, Physics, English, History, Art), following $X = AF + \varepsilon$ with $X_1, \ldots, X_6$ the (centred) subject columns.

```
import numpy as np
import pandas as pd
np.set_printoptions(precision=4, suppress=True)

subjects = [r'Math-1 ($X_1$)', r'Math-2 ($X_2$)', r'Physics ($X_3$)',
            r'English ($X_4$)', r'History ($X_5$)', r'Art ($X_6$)']
marks = [
    [85, 80, 83, 60, 62, 58],
    [78, 75, 80, 55, 60, 59],
    [92, 88, 90, 65, 68, 66],
    [60, 62, 58, 82, 80, 78],
    [63, 60, 65, 85, 83, 80],
    [70, 68, 72, 70, 68, 66],
    [88, 85, 87, 64, 66, 63],
    [55, 52, 50, 90, 88, 86],
    [72, 70, 74, 68, 66, 64],
    [66, 64, 67, 75, 74, 72]
]
X = np.array(marks).T   # rows = subjects, columns = students
n, N = X.shape
print('Number of subjects (n):', n)
print('Sample size (N):', N)
```

```
Number of subjects (n): 6
Sample size (N): 10
```

Note the transpose: rows are the $n = 6$ *variables* $X_1, \ldots, X_6$ and columns are the $N = 10$ observations (students) — matching the $X = AF + \varepsilon$ convention used throughout.

## Computational Illustration — Centring and $S$

As in the theory, we centre each variable and estimate $\Sigma$ by the sample covariance matrix $S$, with entries

$$S_{ij} = \frac{1}{N-1}\sum_{k=1}^{N}(x_{i,k} - \bar{x}_i)(x_{j,k} - \bar{x}_j).$$

```
means = np.mean(X, axis=1)
print('Means:', np.round(means, 2))

Xc = np.array([X[i] - means[i] for i in range(n)])
S = Xc @ Xc.T / (N - 1)
print('\nCovariance matrix S:\n', np.round(S, 3))
```

```
Means: [72.9 70.4 72.6 71.4 71.5 69.2]

Covariance matrix S:
 [[ 156.322  143.156  157.844 -119.733  -93.833  -96.533]
 [ 143.156  133.378  144.956 -110.733  -87.333  -89.422]
 [ 157.844  144.956  165.378 -128.933 -103.111 -105.356]
 [-119.733 -110.733 -128.933  129.378  105.111  105.133]
 [ -93.833  -87.333 -103.111  105.111   87.833   88.   ]
 [ -96.533  -89.422 -105.356  105.133   88.      88.844]]
```

Notice the block structure: high positive covariance among the three STEM subjects and among the three Humanities subjects, and negative covariance *between* the two groups — exactly the pattern a 2-factor model (literacy/numeracy-style split, here "STEM" vs "Humanities") should explain.

## Computational Illustration — Eigendecomposition of $S$

```
eigvals, eigvecs = np.linalg.eigh(S)
idx = np.argsort(eigvals)[::-1]
eigvals, eigvecs = eigvals[idx], eigvecs[:, idx]
print('Eigenvalues (desc):', np.round(eigvals, 3))

explained = eigvals / eigvals.sum()
print('Explained variance ratios:', np.round(explained, 4))
```

```
Eigenvalues (desc): [699.124  55.348   3.4     2.056   1.082   0.124]
Explained variance ratios: [0.9185 0.0727 0.0045 0.0027 0.0014 0.0002]
```

Two eigenvalues clearly dominate (91.9% and 7.3% of total variance — over 99% combined), confirming the two-factor (STEM/Humanities) structure visible in $S$ itself.

As a cross-check, the **Kaiser criterion** (Remark 10.2, above) applied to the *correlation* matrix gives eigenvalues $5.484,\ 0.466,\ 0.024, \ldots$ — only one exceeds 1, but since it alone does not separate the two known groups, and the scree plot (eigenvalues of $S$) shows a sharp elbow after the second value, $m = 2$ is the sensible choice here. This illustrates that Remark 10.2's criteria are guidelines to be combined with judgement, not a mechanical rule — exactly the point Ch-10.qmd itself makes ("a good FA makes sense, a bad one does not").

## Computational Illustration — Loadings $A_m$

Following $A = V\sqrt{L}$ derived above, but keeping only the top $m = 2$ eigenvalues/eigenvectors ($L_m$, $V_m$):

```
m = 2
Lm = np.diag(eigvals[:m])
Vm = eigvecs[:, :m]
Am = Vm @ np.sqrt(Lm)
print('Loading matrix A_m:\n', np.round(Am, 3))
```

```
Loading matrix A_m:
 [[-12.064  -3.171]
 [-11.138  -2.823]
 [-12.624  -1.972]
 [ 10.792  -3.395]
 [  8.7    -3.447]
 [  8.833  -3.166]]
```

Reading off the signs and sizes: Math-1, Math-2 and Physics load strongly (in magnitude) on Factor 1 and weakly on Factor 2, while English, History and Art load the other way round — the same literacy/numeracy-style split as Ch-10.qmd's school-children example, recovered here entirely from simulated numbers rather than given as a fait accompli.

## Computational Illustration — Reconstructing $X$ and $F$

Because $AA^T \approx S = V\sqrt{L}\sqrt{L}V^T$, truncating to $m$ factors gives the low-rank approximation $\hat{X} := V_m V_m^T X$ — the same idea as the theoretical decomposition, now used to check how much information 2 factors retain:

```
Xc_hat = Vm @ Vm.T @ Xc
X_hat = np.array([Xc_hat[i] + means[i] for i in range(n)])
print('Student 1 original:     ', X[:, 0])
print('Student 1 reconstructed:', np.round(X_hat[:, 0], 2))
print('Max reconstruction error:', np.round(np.max(np.abs(X - X_hat)), 2), 'marks')
```

```
Student 1 original:      [85 80 83 60 62 58]
Student 1 reconstructed: [83.59 80.31 84.29 59.48 61.63 59.3 ]
Max reconstruction error: 2.98 marks
```

The reconstruction is close (worst single-mark error under 3, out of scores in the 50s-90s) — consistent with the two factors already explaining over 99% of the variance.

Symmetrically, since $X \approx AF \Rightarrow F \approx A^{-1}X = (V\sqrt{L})^{-1}X = L^{-1/2}V^T X$, we can recover approximate **factor scores**:

$$\hat{F} = L_m^{-1/2} V_m^T \hat{X}.$$

```
F_hat = np.diag(eigvals[:m] ** (-0.5)) @ Vm.T @ Xc_hat
print('Factor 1 - mean, std:', np.round(F_hat[0].mean(), 3), np.round(F_hat[0].std(), 3))
print('Factor 2 - mean, std:', np.round(F_hat[1].mean(), 3), np.round(F_hat[1].std(), 3))
```

```
Factor 1 - mean, std: -0.0 0.949
Factor 2 - mean, std: 0.0 0.949
```

As required by the model assumptions, both estimated factors come out with mean $\approx 0$ and (sample) standard deviation $\approx 1$ — a good internal consistency check that the whole pipeline (from raw marks to $\hat{A}$ and $\hat{F}$) is behaving exactly as the theory predicts.

## Computational Illustration — Geometric Picture

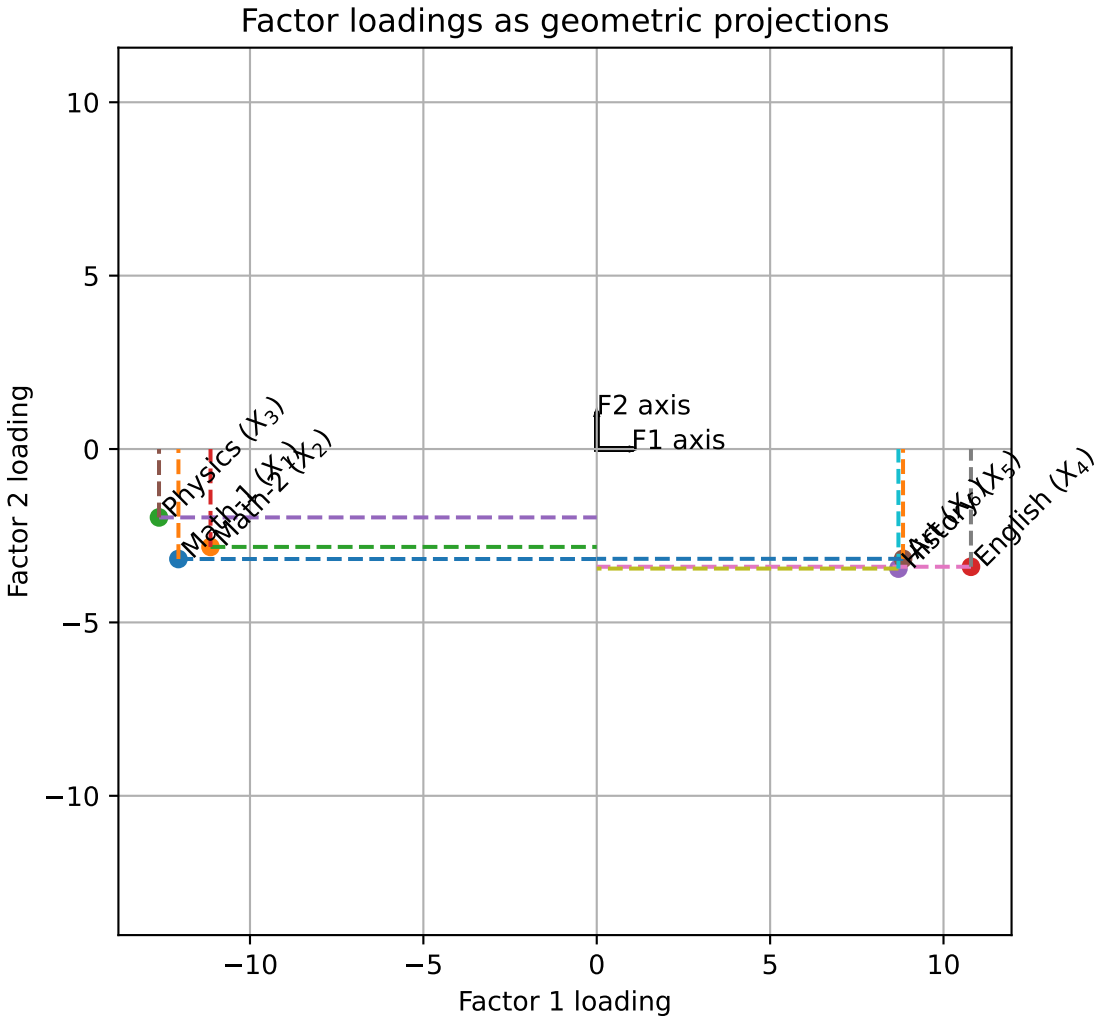
Each row of $A_m$ is a point $(a_{i1}, a_{i2})$ in the "factor-loading plane" — plotting these gives a geometric picture of which variables belong to which factor:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 6))
plt.arrow(0, 0, 1, 0, linewidth=2)
plt.text(1, 0, 'F1 axis')
plt.arrow(0, 0, 0, 1, linewidth=2)
plt.text(0, 1, 'F2 axis')

for i, subj in enumerate(subjects):
    x, y = Am[i, 0], Am[i, 1]
    plt.scatter(x, y)
    plt.text(x, y, subj, rotation=45)
    plt.plot([x, 0], [y, y], '--')
    plt.plot([x, x], [y, 0], '--')

plt.xlabel('Factor 1 loading')
plt.ylabel('Factor 2 loading')
plt.title('Factor loadings as geometric projections')
plt.grid(True)
plt.axis('equal')
plt.show()
```


Factor loadings as geometric projections
Factor 2 loading
Factor 1 loading
F2 axis
F1 axis
Physics ($X_3$)
Math-1 ($X_1$)
Math-2 ($X_2$)
History ($X_5$)
English ($X_4$)
10
5
0
−5
−10
−10
−5
0
5
10

## Computational Illustration — A Real Dataset (BFI)

The notebook repeats the same pipeline on a real personality-questionnaire dataset (BFI: 6-point Likert-scale responses to Big-Five personality items), to show the method at a realistic scale (many variables, real — not simulated — data), selecting $m$ via the Kaiser rule on the **correlation** matrix (Remark 10.2) rather than fixing $m = 2$ by hand:

```
# Requires the accompanying bfi.csv file (not distributed with the
# notebook in this session, hence eval: false - code is included for
# completeness, no fabricated output is shown; see Suggestions-Ch-10.md).
df = pd.read_csv("bfi.csv")
df.drop(['rownames', 'gender', 'education', 'age'], axis=1, inplace=True)
df.dropna(inplace=True)

X = np.array(df).T
n, N = X.shape
means = np.mean(X, axis=1)
Xc = np.array([X[i] - means[i] for i in range(n)])
S = Xc @ Xc.T / (N - 1)
std = np.sqrt(np.diag(S))
r = S / np.outer(std, std)

eigvalsR, eigvecsR = np.linalg.eigh(r)
idx = np.argsort(eigvalsR)[::-1]
eigvalsR, eigvecsR = eigvalsR[idx], eigvecsR[:, idx]

m = sum(eigvalsR > 1)          # Kaiser criterion, applied automatically
Lm = np.diag(eigvalsR[:m])
Vm = eigvecsR[:, :m]
Am = Vm @ np.sqrt(Lm)

fa = pd.DataFrame(Am, columns=[f"Factor {i+1}" for i in range(m)])
fa.insert(0, "Characteristics", df.columns)
for j in range(1, m + 1):
    print(fa.sort_values(by=f"Factor {j}", key=abs, ascending=False))
```

Sorting each factor's column by absolute loading (rather than reading the raw matrix) is itself a practical technique worth noting: with dozens of items it is the only way to see, at a glance, which items define each factor — this is exactly how the school-children and ski-resort examples' factors were "named" in Ch-10.qmd, done here programmatically instead of by eye.

# Summary

Lecture 1's theoretical arc: $X = AF + \varepsilon \rightarrow$ assumptions $\rightarrow \Sigma = AA^T + \Psi \rightarrow R = AA^T = \Sigma - \Psi = VLV^T \rightarrow A = V\sqrt{L} \rightarrow$ non-uniqueness of $A$ under rotation.

The notebook makes every one of these steps computable: simulated $X \rightarrow$ sample $S \rightarrow$ eigendecomposition $\rightarrow$ truncated $A_m = V_m\sqrt{L_m} \rightarrow$ reconstructions $\hat{X}, \hat{F} \rightarrow$ a geometric picture of the loadings $\rightarrow$ the same pipeline on real data.

What is *not* covered here — the Kaiser criterion beyond its single bullet, scree plots, communality, factor-loading thresholds, orthogonal/oblique rotation formulas (Varimax, Quartimax, Equamax, Direct Oblimin, Promax), PCA, and "PCA vs FA" — is genuine Ch-10.qmd content with no counterpart in the handwritten Lecture-1 notes.